

Homework 3

Computational Bootcamp, Summer 2026

Due: Wednesday, August 19

Submission: you will be added as a collaborator on your own private repository `hw3-<your-github-username>` in the course organization. Clone it, write your answers in `homework3.jl`, and `git push` to submit. Your results will be posted back as a GitHub issue. See the repository README for setup and the required function names.

Question 1: Parallelization and the speed of convergence

This question reuses the deterministic growth model we have discussed several times: log utility, $\alpha = 0.36$, $\delta = 0.025$:

$$V(k) = \max_{k'} \{ \log(c) + \beta V(k') \}, \quad c = k^\alpha + (1 - \delta)k - k'.$$

Solving it once is cheap, but solving it for sixty discount factors is not. Those sixty solves never need to talk to each other, so they are “embarrassingly parallel”.

- A. Write `solve_growth( $\beta$ )`, returning the capital grid and the policy $k'(k)$, and `steady_state( $\beta$ )`, returning k_{ss} , found numerically by finding a fixed point of the policy function $k' = k$ or $g(k) = k$. You can do this using a root finder on the interpolated policy function ($g(k) - k = 0$). Check it against the closed form $k_{ss} = ((1/\beta - 1 + \delta)/\alpha)^{1/(\alpha-1)}$ from the Euler equation. Note $k_{ss} \approx 49$ at $\beta = 0.995$, so your capital grid has to reach further than we had previously.
- B. Then write `half_life( $\beta$ )`: start at $k_0 = k_{ss}/2$, iterate $k_{t+1} = k'(k_t)$, and return the number of periods needed to close half the gap to k_{ss} . Interpolate the policy so capital is not confined to the grid.
- C. Run `half_life` over 60 values of $\beta \in [0.90, 0.995]$, first with a plain `for` loop and then parallelized with `Distributed`. Check that the parallel answers match the serial ones, and report times and speedups at 1, 2, 4, and 8 cores (if you have 8 available). Put `@everywhere` on the definitions, not on your timing code. Plot the half-life against β : is a more patient economy quicker or slower to converge? (Predict first.) And why did your speedup fall short of your worker count?

Optional, ungraded: if you have an SSCC account, re-run part C on Linstat with 20 workers (`julia -p 20`) and report what you get.

Question 2: Job search with an expiring benefit

An unemployed worker draws one wage offer w each period, iid from a known distribution: 101 values evenly spaced on $[10, 60]$, each equally likely. Accepting means keeping that wage forever,

worth $w/(1 - \beta)$; rejecting means collecting benefit b this period and drawing again next period. Let $\beta = 0.98$, $b = 25$, and let the benefit last $T = 40$ periods, after which the worker must accept whatever is offered, so $V_{T+1}(w) = w/(1 - \beta)$. For $t \leq T$,

$$V_t(w) = \max \left\{ \underbrace{\frac{w}{1 - \beta}}_{\text{accept}}, \underbrace{b + \beta \mathbb{E}_{w'}[V_{t+1}(w')]}_{\text{reject}} \right\}.$$

This is a finite-horizon problem: the terminal condition is known, so there is no fixed point to find and you sweep backward once.

- A. The value of rejecting, $R_t = b + \beta \mathbb{E}[V_{t+1}(w')]$, does not depend on today's offer, so the worker accepts if and only if $w \geq \bar{w}_t \equiv (1 - \beta)R_t$: one *reservation wage* per period. Write `reservation_path(T;  $\beta = 0.98$ ,  $b = 25.0$ )`, computing $R_T, \dots, R_1$ by backward induction and returning $(\bar{w}_1, \dots, \bar{w}_T)$.
- B. Plot $\bar{w}_t$ against t and explain the shape, especially the last few periods before the benefit runs out. (Empirically, job-finding rates spike around benefit exhaustion.)
- C. Now solve the *infinite*-horizon version, where the benefit never expires, as `reservation_infinite( $\beta = 0.98$ ,  $b = 25.0$ )`: R solves the scalar fixed point $R = b + \beta \sum_j p_j \max\{w_j/(1 - \beta), R\}$. You can use a root finder to solve for R . Compare $\bar{w}_1$ from part A to it as you raise T through 10, 40, 100, 400 and report the gap. Finally, recompute the infinite-horizon reservation wage for $b \in \{10, 20, 30, 40\}$ and report the implied $P(\text{accept})$ and expected unemployment duration $1/P(\text{accept})$. What is the policy trade-off?

Question 3: Endowment economy — Howard iteration and the endogenous grid method

An infinitely lived household with CRRA preferences $u(c) = c^{1-\gamma}/(1 - \gamma)$, $\gamma = 2$, receives a stochastic endowment y and saves at a fixed gross return $1 + r$, $r = 0.03$. It cannot borrow ($a' \geq 0$), and $\beta = 0.96$. The endowment takes two values, $y_h = 1.0$ and $y_l = 0.5$, with $\Pi = \begin{bmatrix} 0.95 & 0.05 \\ 0.75 & 0.25 \end{bmatrix}$, where $\Pi_{ij} = P(y' = y_j \mid y = y_i)$. The problem is

$$V(a, y) = \max_{a' \geq 0} \left\{ u(c) + \beta \sum_{y'} \Pi(y' \mid y) V(a', y') \right\}, \quad c = (1 + r)a + y - a'.$$

This is the household block of an incomplete-markets model (Huggett, Aiyagari), where in equilibrium r clears the asset market. Use an asset grid of 101 points from 0 to 5 (the first point *is* the borrowing limit) and a tolerance of 10^{-5} .

- A. Solve by value function iteration, interpolating the value function and choosing a' continuously with an optimizer: `solve_vfi(; m = 0)`. Return V , the policy $a'(a, y)$, and the iteration count. Report iterations and run time.
- B. *Howard iteration*. Most of the cost is the maximization, and the policy converges long before the value does. So after each maximizing sweep, keep the policy you just found and apply it m more times without re-maximizing:

$$V(a, y) \leftarrow u(c_g(a, y)) + \beta \sum_{y'} \Pi(y' \mid y) V(g(a, y), y'),$$

where g is the policy and $u(c_g)$ the flow payoff, both saved from the sweep. Implement this as the `m` keyword of `solve_vfi` ($m = 0$ is plain VFI) and report iterations and run time for $m \in \{0, 5, 20, 50, 100\}$. Why does the gain flatten out? What is the floor on the number of maximizing sweeps, and what sets it?

- C. *The endogenous grid method.* Because the budget constraint is linear in a , there is no need for a change of variables. Given c and a' , we can solve for today's assets $a = (c + a' - y)/(1 + r)$. The exogenous grid will be on tomorrow's assets $\{a'_j\}$ and store the policy $c(\cdot, y)$ on those same 101 points. Wherever $a' > 0$ the Euler equation holds with equality:

$$u'(c(a, y)) = \beta(1 + r) \sum_{y'} \Pi(y' | y) u'(c(a', y')),$$

The Euler equation does not hold where the borrowing constraint binds (unconstrained choice $a' < 0$) and the decision is thus $a' := 0$, and $c(a, y) = y + (1 + r)a$. Follow this algorithm to implement the endogenous grid method in this environment:

- (a) For each y and fixed assets tomorrow a'_j , calculate the right hand side of the Euler equation $\text{RHS}_j = \beta(1 + r) \sum_{y'} \Pi(y' | y) u'(c_j(y'))$, then invert in closed form: $c_j^{\text{endog}}(y) = (\text{RHS}_j)^{-1/\gamma}$.
- (b) Retrieve today's state from the budget constraint, $a_j^{\text{endog}}(y) = (c_j^{\text{endog}}(y) + a'_j - y)/(1 + r)$: the *endogenous* grid of assets today, which differs across y .
- (c) Interpolate $(a_j^{\text{endog}}(y), c_j^{\text{endog}}(y))$ back onto the fixed grid.
- (d) You now have a set of points $(a_j, y, c_j^{\text{next}}(y))$. However, you must check for where these points violate the borrowing constraint and correct the policy functions. Wherever the implied $a' = y + (1 + r)a_j - c_j(y) < 0$, we overwrite this by hand as $a' := 0$. Then, we can recover the true consumption by $c_j^{\text{next}}(y) = y + (1 + r)a_j$. Note that this overwriting puts a kink in this iteration's policy wherever it fires.
- (e) Repeat iterating until $\max_{j,y} |c_j^{\text{next}} - c_j| < \text{tol} = 10^{-5}$ on the fixed grid. Consuming everything, $c = (1 + r)a + y$, is a fine initial guess.

Write the loop above as `solve_egm()`, which solves for a fixed point in the consumption policy function. Recover the savings policy from the budget constraint as $a'_{EGM}(a, y) = (1 + r)a + y - c(a, y)$. Plot the savings policy over your part-A policy and report the maximum absolute difference and the run time against parts A and B.

Finally, report for each income state the threshold $a^*(y)$ below which the borrowing constraint binds, if it does. Where the constraint does bind, the two policies must agree exactly (both are pinned to $a' = 0$). They will disagree most just above $a^*(y)$. Which do you trust near the kink?